Experimental Dialog Correlation and Recovery Across Agent Adapters

Proposed Structure

Team Code: A3-PS021-TC047

Team Name: 404 Crew

AIORI-3 Hackathon • 6G & Future Networks

Mentor:

Bharani H, bharani1@srmist.edu.in

Students:

Subehaa SB, us4373@srmist.edu.in

Prem J, pj6631@srmist.edu.in

AIORI Hackathon 3

Contents

1. Problem Description

2. Background

3. Solution Proposed

4. Optimizations Proposed by the Team

5. Solution Architecture and Design

6. Timeline of Delivery

7. References

8. Conclusion

Pseudocode for the Mentor Demonstration

Pseudocode for the Mentor Demonstration

The short flow below shows the main retry, recovery and duplicate-request logic that will be demonstrated to the mentor.

```
ADAPTER A (initiator, Spring Boot)
  CREATE TASK:
    GENERATE dialog_id, task_id, request_id
    SAVE task with state = CREATED
    SEND request to Adapter B
  IF TIMEOUT OR REPLY LOST OR FAILURE:
    RETRY with the SAME dialog_id, task_id, request_id (attempt + 1)
    IF no attempts are left: MARK task FAILED
  ON RESTART OF ADAPTER A:
    LOAD all tasks that are not finished
    RESEND each one with its SAME IDs

ADAPTER B (executor, FastAPI)
  RECEIVE REQUEST:
    CHECK request against the shared schema
    TRY TO CLAIM (dialog_id, request_id) AS PROCESSING   (atomic, unique key)
    IF THE CLAIM ALREADY EXISTS:
      IF payload hash differs:       REJECT AS CONFLICT
      ELSE IF status is COMPLETED:   RETURN stored result, duplicate = true
      ELSE (still PROCESSING):       RETURN in-progress answer
    ELSE:
      RUN worker (stub, or optional Hugging Face agent)
      IN ONE TRANSACTION: WRITE effect AND MARK request COMPLETED
      RETURN result, duplicate = false
  ON RESTART OF ADAPTER B:
    LOAD saved requests
    HANDLE claims left in PROCESSING   (rule to be finalised with the mentor)
```

1. Problem Description

When two agents are working on the same task, a restart or a dropped connection can make the next request look like a new one. That can leave the task half-finished or cause the same operation to happen twice. Our prototype will keep the task identifiable from start to finish, even when one adapter goes down and comes back.

- Connect two mock agent adapters: a Spring Boot initiator (Adapter A) and a FastAPI executor (Adapter B).
- Give each task a dialog ID, task ID and request ID, created once and reused on every retry.
- Keep task state in a database so it survives an adapter restart.
- Recognise a request that has already been processed and return its stored result instead of running the operation again.
- Test five disconnect and retry situations.
- Allow only the state changes defined for the task.

2. Background

AI agents are increasingly being used to handle tasks that involve multiple steps and communication between different services. In many cases, a task may pass between agents built using different frameworks or run on separate services. For such systems to work reliably, they need to keep track of the task even when an agent becomes temporarily unavailable.

A problem occurs when an agent disconnects, restarts, or fails to respond while a task is in progress. When communication resumes, the system may lose track of the earlier conversation or process the same request again. This can lead to duplicated operations, incomplete tasks, or inconsistent task states. These issues become more difficult to handle when multiple agents are involved.

The proposed project, PS-021, focuses on improving how dialog information and task identity are maintained across agent adapters. The system will use dialog IDs, task IDs, request tracking, and saved state to identify ongoing work. If an agent disconnects or restarts, the system should be able to restore the saved state and continue the same task. It will also check whether a request has already been processed to avoid unnecessary duplicate execution.

The team plans to demonstrate this approach using two mock agent adapters and a set of simulated disconnect and retry scenarios. The aim is to show how basic state management, request deduplication, and recovery mechanisms can make communication between agents more reliable.

3. Solution Proposed

We will build a small system around two mock agent adapters that talk to each other directly. Each adapter keeps its own saved state, so neither depends on a separate central manager.

- **Adapter A (Spring Boot, initiator):** creates the dialog, task and request IDs, saves the task before sending it, sends it to Adapter B, tracks the task lifecycle, retries with the same IDs after a timeout or a lost reply, and after its own restart reloads unfinished tasks and resends them.
- **Adapter B (FastAPI, executor):** validates each request, claims the (dialog ID, request ID) pair in the database so only one attempt can run, detects repeated requests, runs the task through a worker, saves the effect and marks the request completed in one transaction, and replies, marking replayed answers as duplicates.

- **Worker:** a deterministic stub worker by default, used for all tests. An optional Hugging Face agent can be plugged in for the demo. The worker only produces content; recovery decisions stay in our backend code.
- **Shared contract:** one schema for requests and responses that both adapters check.

What the prototype will show

- Adapter A sends a task to Adapter B.
- The task keeps the same dialog and task IDs throughout the exchange.
- A simulated failure interrupts the exchange (Adapter B stops, B's reply is lost, or Adapter A restarts).
- The interrupted adapter restarts, reloads its saved state and resends with the same IDs.
- A retry with an already processed request ID gets the stored result back and the operation is not run again; a test confirms that exactly one effect was recorded.
- Only the state changes defined for the task are accepted.

Scope and limits

This is an experimental prototype. The dialog-state schema is labelled experimental, is inspired by emerging IETF agent-protocol discussions [7], and will be aligned with the schema the mentor selects. We do not claim a final IETF-standard implementation, complete MCP/A2A interoperability, or exactly-once delivery in every failure condition. Our claim is narrower: for the mock operation in this prototype (a database write), a retry does not repeat the effect in the failure scenarios we test.

4. Optimizations Proposed by the Team

The main improvement is to make failure and retry part of the normal workflow. Instead of starting a new task after a restart, the system will find the saved task and continue from its last known state.

Area	Without our mechanism	With our approach
Task identity	A retry may look like a new task.	Dialog ID and task ID stay with the task and are reused on every retry.
Restart	The current state may be lost.	Saved state is loaded again by the adapter that restarted.
Duplicate request	The same operation may run twice.	The request ID is claimed atomically; a repeat gets the stored result instead of running again.
Two identical requests at once	Both may pass a simple check and both run.	The first claim wins; the other receives the stored or in-progress answer.
Crash after the work, before it is recorded	The work is done but not recorded, so a retry repeats it.	The effect and the completed mark are saved in one transaction.
Changed content under the same ID	It may be treated as the same request.	A payload hash check rejects it as a conflict.
Failure	Recovery is mostly manual.	Disconnect and retry cases are tested directly.
State changes	Unexpected transitions can slip through.	Only defined transitions are accepted.

The demonstration will show a normal request next to the same request arriving again after a failure.

5. Solution Architecture and Design

The first version will run locally so the whole system can be developed and demonstrated without depending on paid cloud infrastructure.

Component	Role
Client / test script	Starts tasks, polls the status by task ID, and injects the failures.
Adapter A (Spring Boot)	Initiator: IDs, lifecycle state, retry, restart recovery.
Adapter B (FastAPI)	Executor: validation, atomic claim, duplicate handling, result storage.
Worker	Stub by default (tests); optional Hugging Face agent (demo).
PostgreSQL	One instance with two schemas, adapter_a and adapter_b. Each adapter uses only its own.
Shared contract	One request/response schema that both adapters validate against.
Docker Compose	Runs the services together. Tests run the scenarios over HTTP (pytest).

5.1 Architecture

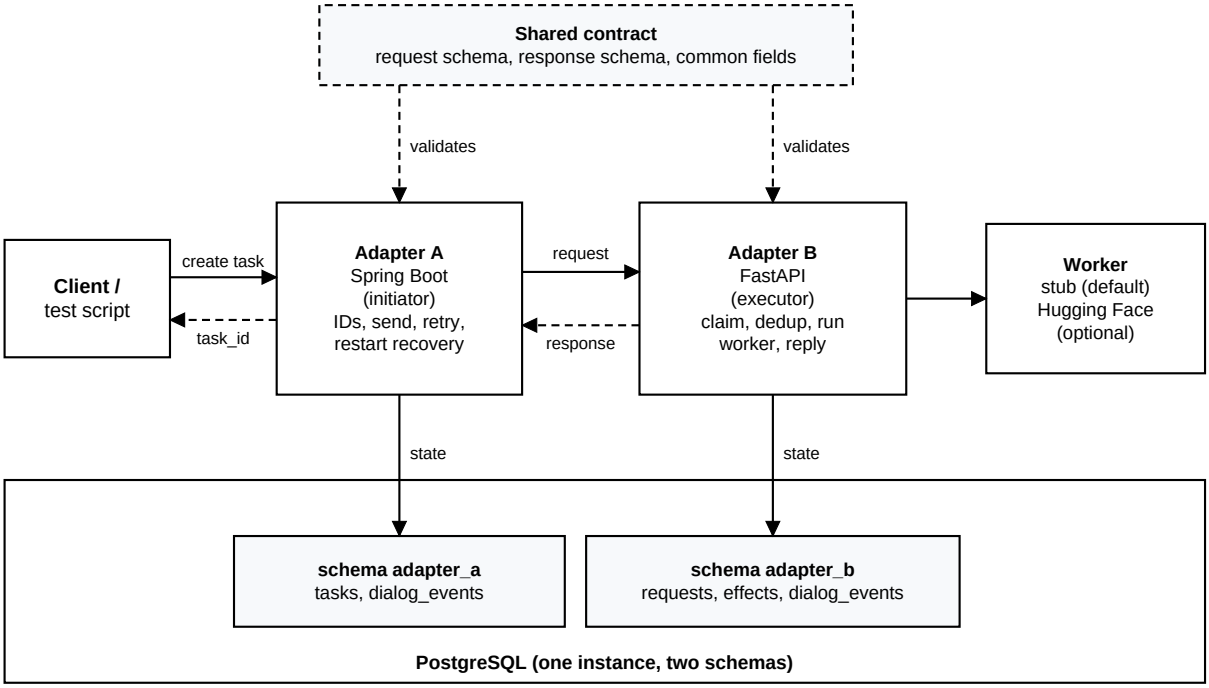


Figure 1. System architecture: two peer adapters, a shared contract and one database with two schemas.

5.2 Complete flow

The flow below follows one task from creation to completion. Figure 2 covers Adapter A, Figure 3 covers Adapter B, and Figure 4 covers recovery after a restart. The duplicate check happens when a request arrives, before any work runs.

- Create task: Adapter A assigns the dialog ID, task ID and request ID, and saves the task.
- Adapter A sends the request to Adapter B.
- Adapter B claims the request, runs it, and saves the result.
- A disconnect, a lost reply or a restart interrupts the exchange.
- The request is retried with the same IDs.
- Adapter B recognises the request ID and returns the stored result; after its own restart, Adapter A resumes the unfinished task.

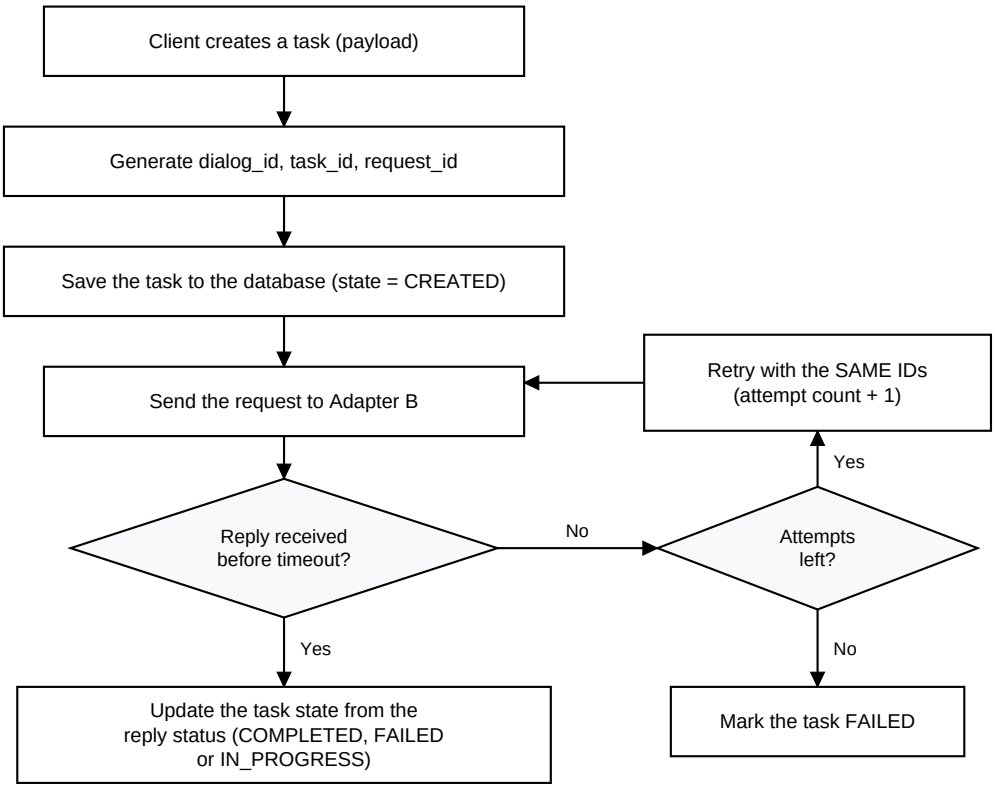


Figure 2. Adapter A: create, send, retry with the same IDs, and update state.

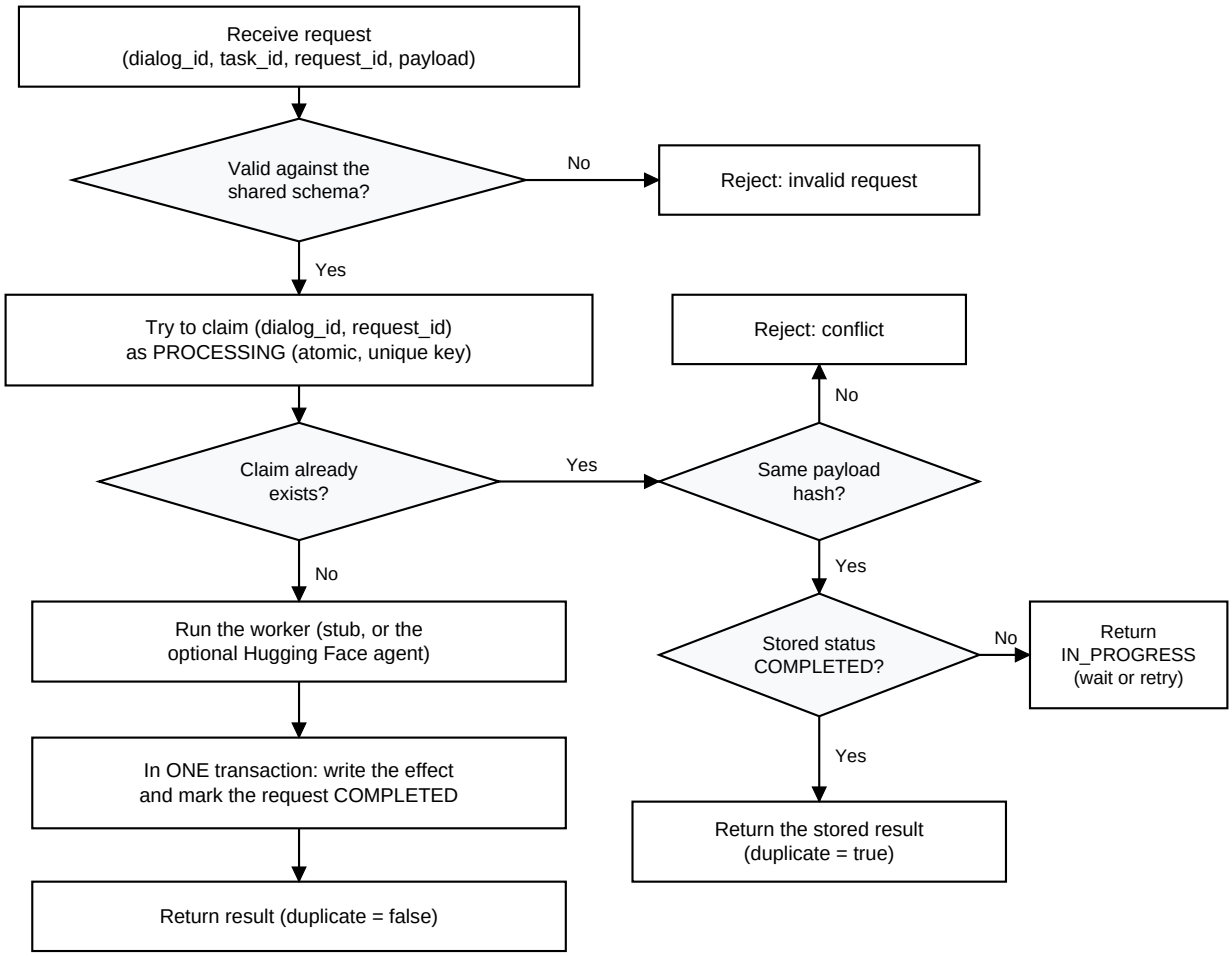
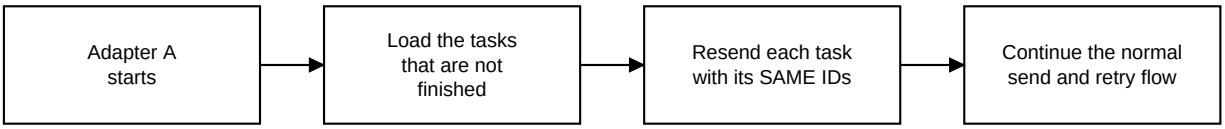


Figure 3. Adapter B: handling a request, including the duplicate and conflict cases.

Adapter A (Spring Boot)



Adapter B (FastAPI)

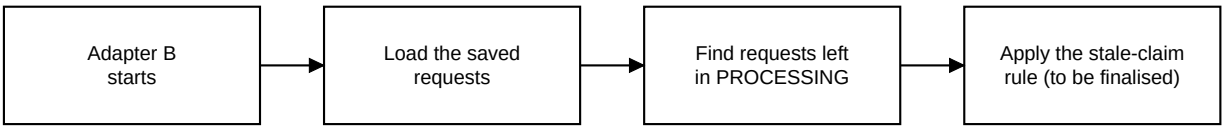


Figure 4. Recovery after a restart on each adapter.

6. Timeline of Delivery

Stage	Work	Output
1. Setup	GitHub repo, project structure, Docker Compose, shared contract draft, transition table	Services start locally
2. Dialog handling	Adapters A and B: IDs, task state, request and response	One task completes end to end
3. Persistence and deduplication	Saved state, atomic claim, payload hash, effect and completed mark in one transaction	Restart recovery; a duplicate returns the stored result
4. Failure tests	Five disconnect/retry cases plus the extra tests	Test results
5. Submission	Report, PDF, pseudocode, demonstration, video, repository ownership transfer	Final submission

7. References

[1] S. Kumar and P. R. Cohen, “Towards a fault-tolerant multi-agent system architecture,” in *Proceedings of the Fourth International Conference on Autonomous Agents*, 2000, pp. 459–466, doi: 10.1145/336595.337570.

[2] J. Ezekiel and A. Lomuscio, “Combining fault injection and model checking to verify fault tolerance, recoverability, and diagnosability in multi-agent systems,” *Information and Computation*, vol. 254, no. 2, pp. 167–194, 2017, doi: 10.1016/j.ic.2016.10.007.

[3] M. Ganzha, M. Paprzycki, C. Badica, R. Unland, D. Mitrović, Z. Budimac, M. Ivanović, and M. Vidaković, “Agent-based approaches to managing fault-tolerant networks of distributed multi-agent systems,” *Multiagent and Grid Systems*, vol. 7, no. 6, pp. 203–218, 2011, doi: 10.3233/MGS2011-0176.

[4] H. van Ditmarsch, K. Fruzsa, R. Kuznets, and U. Schmid, “A logic for repair and state recovery in Byzantine fault-tolerant multi-agent systems,” in *Proc. 12th Int. Joint Conf. on Automated Reasoning (IJCAR)*, 2024, pp. 114–134, doi: 10.1007/978-3-031-63501-4_7.

[5] J.-T. Huang, J. Zhou, T. Jin, X. Zhou, Z. Chen, W. Wang, Y. Yuan, M. R. Lyu, and M. Sap, “On the resilience of LLM-based multi-agent collaboration with faulty agents,” in *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267, 2025, pp. 26202–26226.

[6] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, “Why do multi-agent LLM systems fail?,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025.

[7] IETF, “Agent protocols (agentproto) working group,” IETF Datatracker. [Online]. Available: https://datatracker.ietf.org/group/agentproto/about/

8. Conclusion

The prototype will show two agent adapters working on the same task without losing track of it when a connection drops or an adapter restarts. The main focus is practical recovery: keep the task identity, restore the saved state and make sure a retry does not repeat an operation.

The prototype will be judged by what can be observed: the same task is recognised after an interruption, a repeated request returns the stored result without a second effect, and the five failure scenarios produce documented results. The limits of the approach are stated openly in this document.